

Conception de BD réparties

Fragmentation

2023-2024

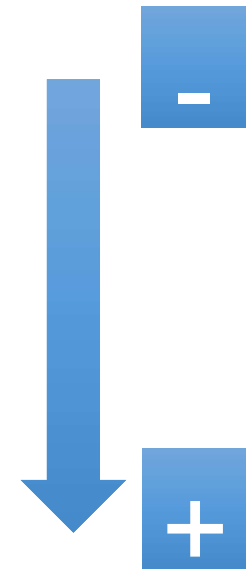
igueye@ept.sn

Bases de Données Réparties

- Définition
- Conception
- Décomposition
- Fragmentation horizontale et verticale
- Réplication
- SGBD répartis hétérogènes

BD réparties (1)

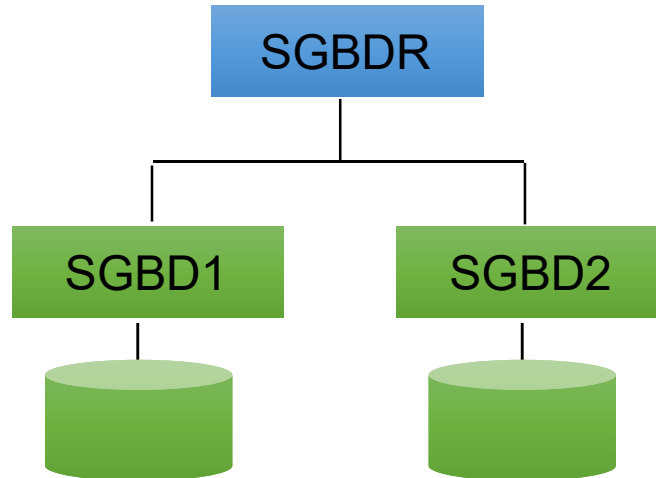
- Principe
 - Un site héberge une BD : accès local rapide, interne au site.
 - Accès global possible à des BD situées sur des sites externes
- Plusieurs niveaux d'intégration :
 - Accès distant (Remote Data Access)
 - Vues réparties
 - Extension du mécanisme de vues
 - Définir des vues sur plusieurs sites.
 - Médiateurs
 - BD réparties/fédérées



BD Réparties (2)

- BD réparties :
 - Plusieurs BD sur plusieurs sites, mais une seule BD « logique ».
 - Fédérée : intègre des bases et des schémas existants
 - Répartie « pur » : conçue répartie. Pas d'accès locaux
- Les ordinateurs (appelés **sites**) communiquent via le réseau et sont faiblement couplés
 - pas de partage de MC, disque, au contraire de *BD parallèles*
- Chaque **site**
 - contient des données de la base,
 - peut exécuter des transactions/requêtes **locales** et
 - **participer** à l'exécution de transactions/requêtes **globales**

SGBD réparti

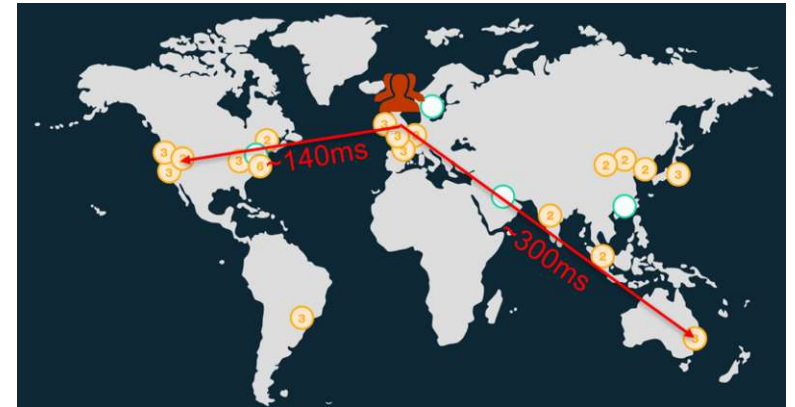


Rend la répartition (ou distribution) *transparente*

- dictionnaire des données (catalogue, métabase) réparties
- traitement des requêtes réparties
- gestion de transactions réparties
- gestion de la cohérence et de la sécurité

Paramètres à considérer

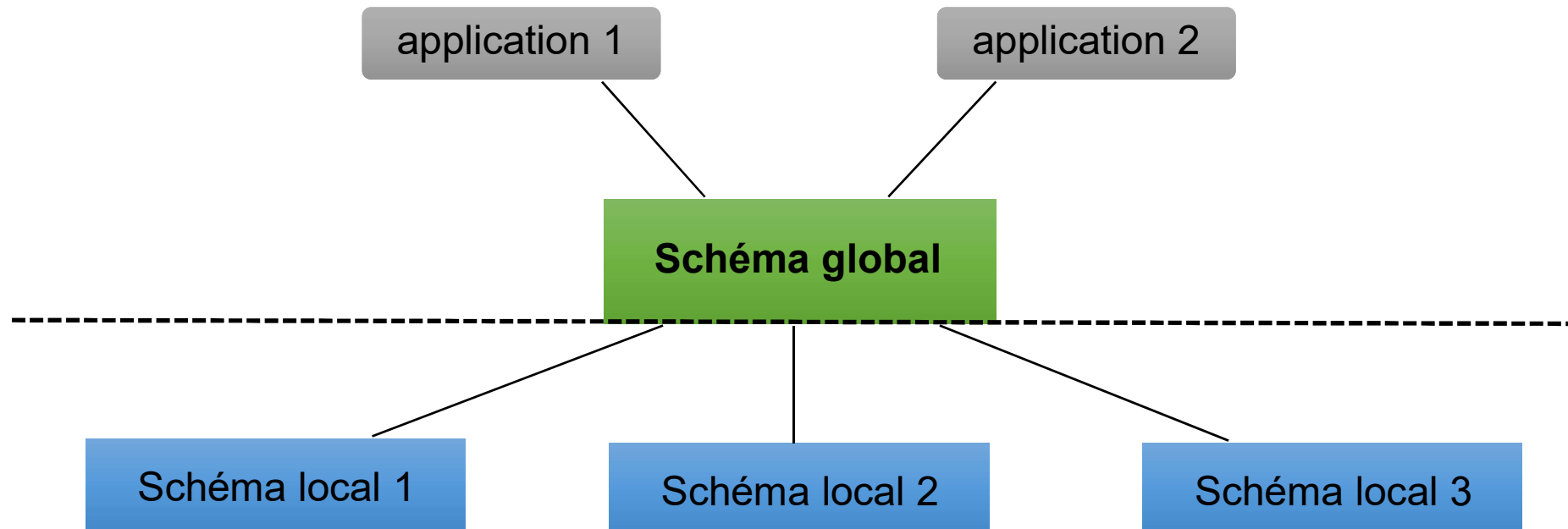
- Coût et **temps de communication** entre deux sites
 - Accès réseau (longue distance, WAN, MAN) beaucoup plus coûteux que accès disque
- Fiabilité : fréquence des pannes
 - des sites, du réseau (cf. P2P)
- Accessibilité aux données
 - accès aux données en cas de panne des sites, du réseau.
 - accès aux sites les moins encombrés, les plus puissants
- A lire : <https://read.acloud.guru/why-and-how-do-we-build-a-multi-region-active-active-architecture-6d81acb7d208>



Evaluation de l'approche BDR

- avantages
 - extensibilité
 - partage des données hétérogènes et réparties
 - performances avec le parallélisme
 - Disponibilité et localité avec la réplication
- inconvénients
 - administration complexe
 - complexité de mise en œuvre
 - distribution du contrôle
 - surcharge (l'échange de messages augmente le temps de calcul)

Architecture de schémas



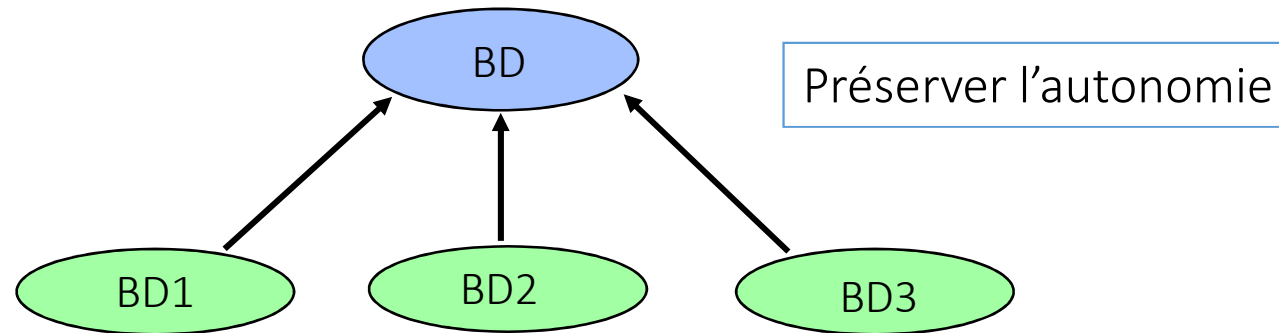
- indépendance applications / bases locales
- schéma global lourd à gérer

Schéma global

- Schéma conceptuel global
 - description globale et unifiée de toutes les données de la BDR
 - Nom des relations avec leurs attributs
 - Fournir l'indépendance à la répartition
- Schéma de placement
 - règles de correspondance avec les données locales
 - Vues globales définies sur les relations locales (cf. global as view)
 - Fournit l'indépendance à la localisation, la fragmentation et la duplication
- Le schéma global fait partie du dictionnaire de la BDR et peut être conçu comme une BDR (dupliqué ou fragmenté)

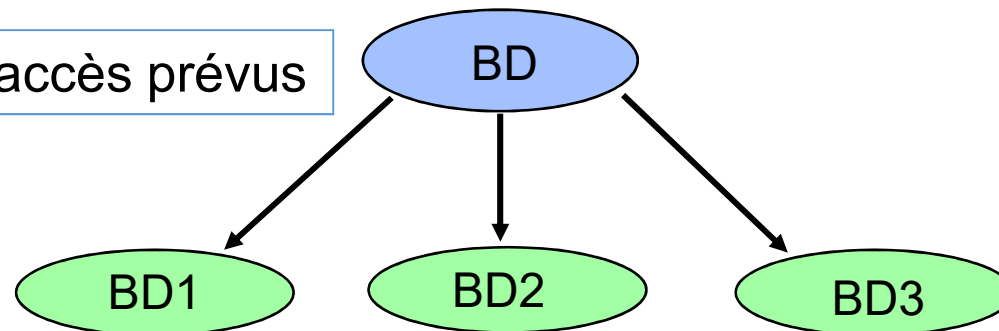
Migration vers une BDR : 2 approches

Intégration logique des BD locales existantes (fédérée, médiateur)

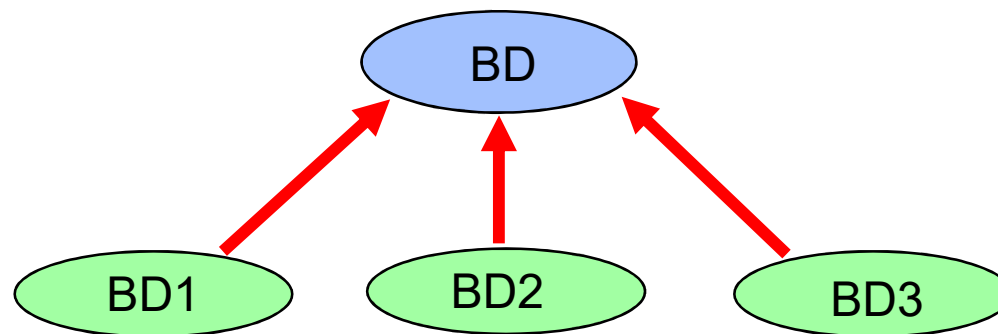


Décomposition en BD locales : répartie « pur »

En fonction des accès prévus

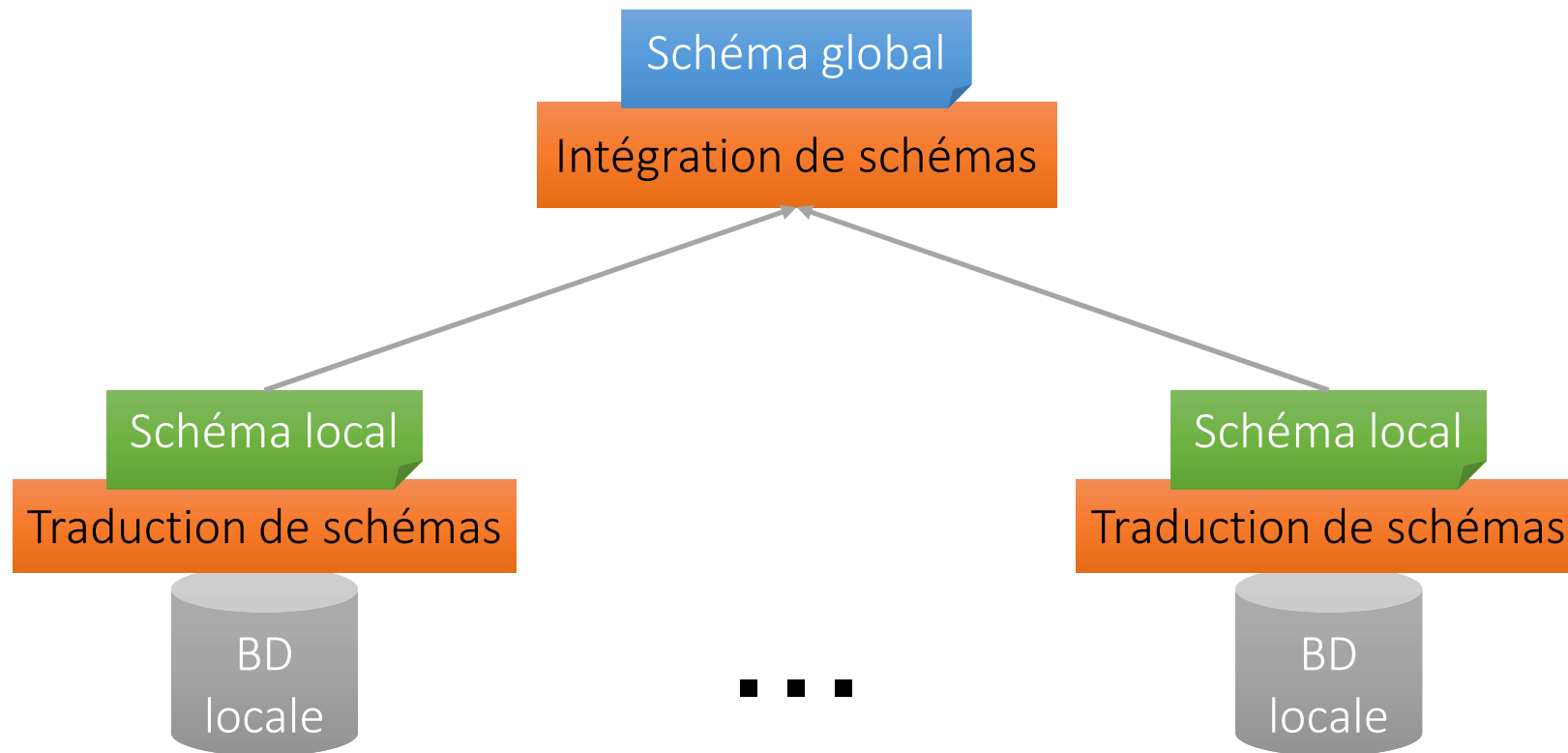


Intégration de BD existantes



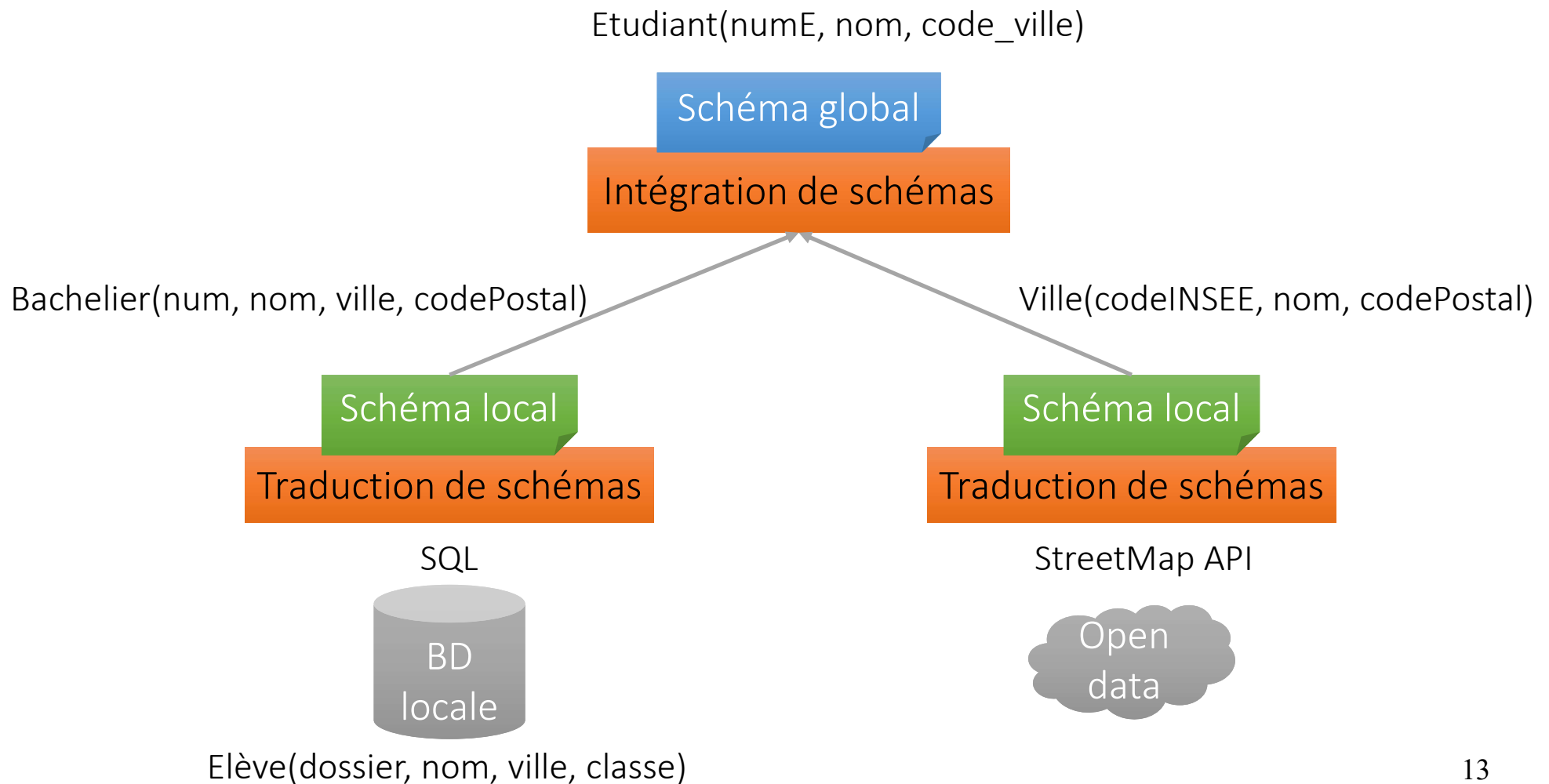
Conception d'une BDR par intégration

Approche médiateur



Conception d'une BDR par intégration

Exemple



Intégration de schémas

1. Pré-intégration

- ❑ Les schémas sont transformés pour les rendre plus homogènes
- ❑ identification des éléments reliés (e.g. domaines équivalents) et établissement des règles de conversion (e.g. 1 inch = 2,54 cm)
- ❑ Pbs : hétérogénéité des modèles de données, des puissances d'expression, des modélisations

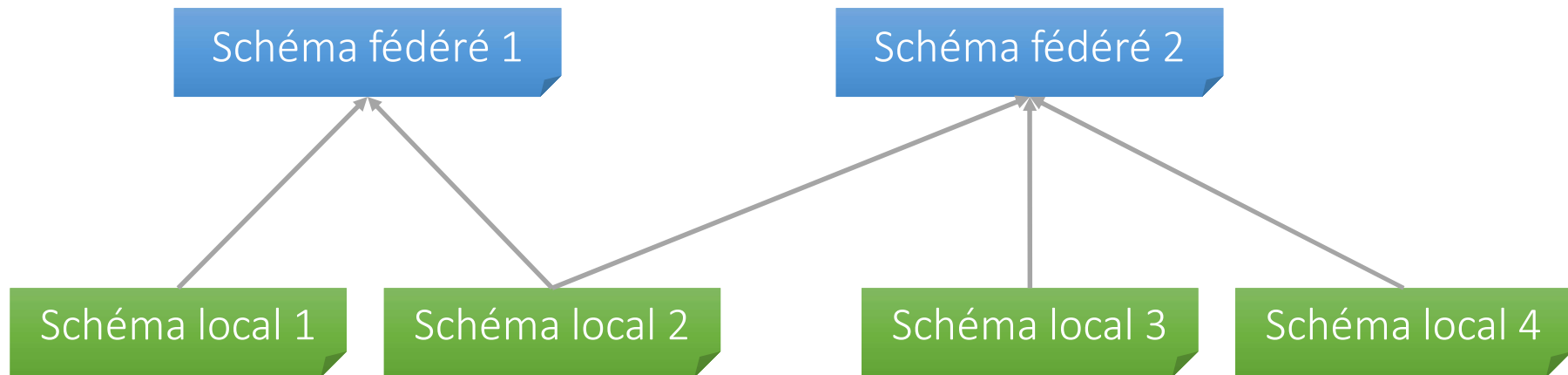
2. Comparaison

- ❑ identification des conflits de noms (synonymes et homonymes) et des conflits structurels (types, clés, dépendances)

3. Conformance

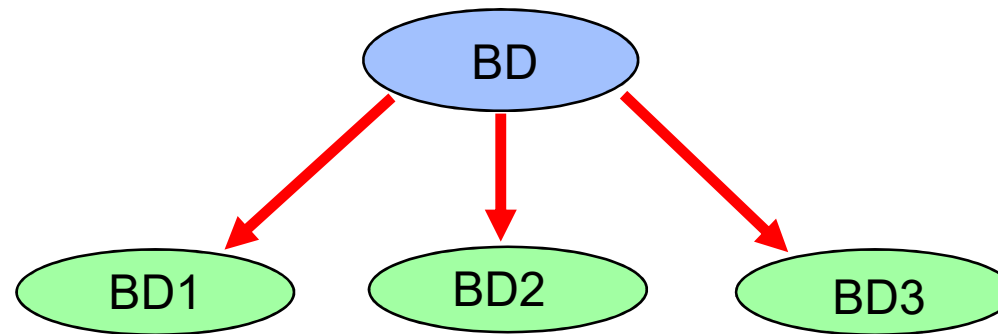
- ❑ résolution des conflits de noms (renommage) et des conflits structurels (changements de clés, tables d'équivalence)
- ❑ Définition de règles de traduction entre le schéma intégré et les schémas initiaux.

Architecture fédérée

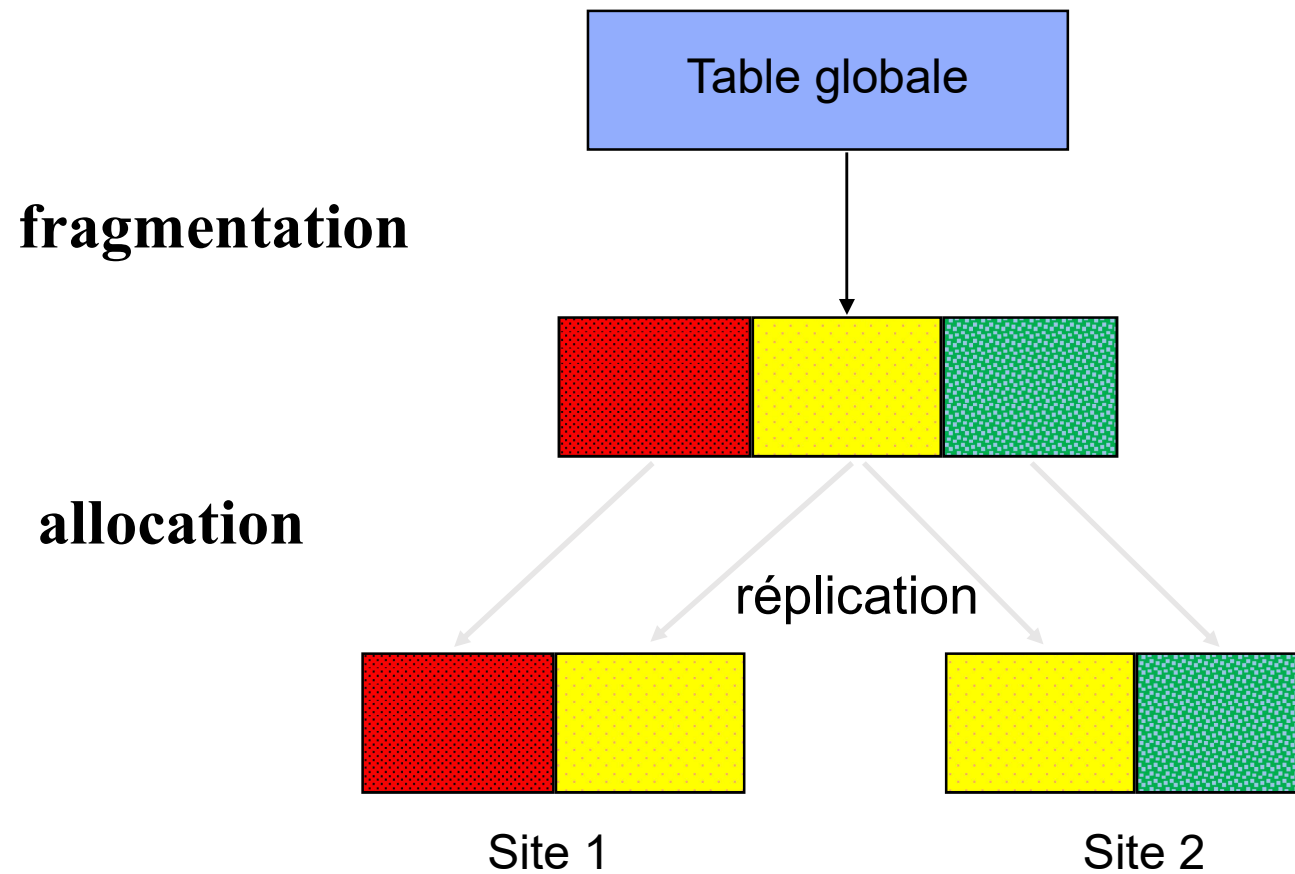


Moyen contrôlé de migration
Plusieurs niveaux possibles

Décomposition



Conception par décomposition



Objectifs de la décomposition

Fragmentation

- Trois types : horizontale, horizontale dérivée, verticale
 - Possibilité de composer plusieurs fragmentations: mixte
- Performances en favorisant les accès (et traitements) locaux
- Equilibrer la charge de travail entre les sites (parallélisme)
- Contrôle de concurrence plus simple pour les accès à un seul fragment

Trop fragmenter : BD éclatée, nombreuses jointures réparties

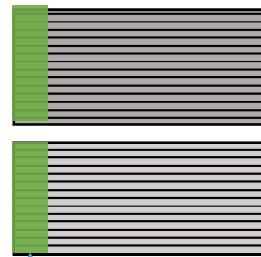
Duplication (ou réplication)

- favoriser les accès locaux
- augmenter la disponibilité des données

Trop répliquer : surcoût de maintenir cohérence des répliques

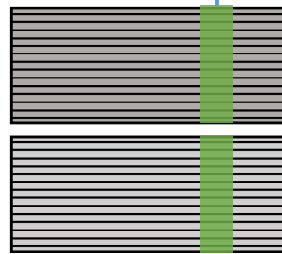
Types de Fragmentation

horizontale



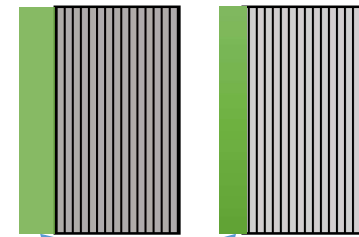
clé

ref



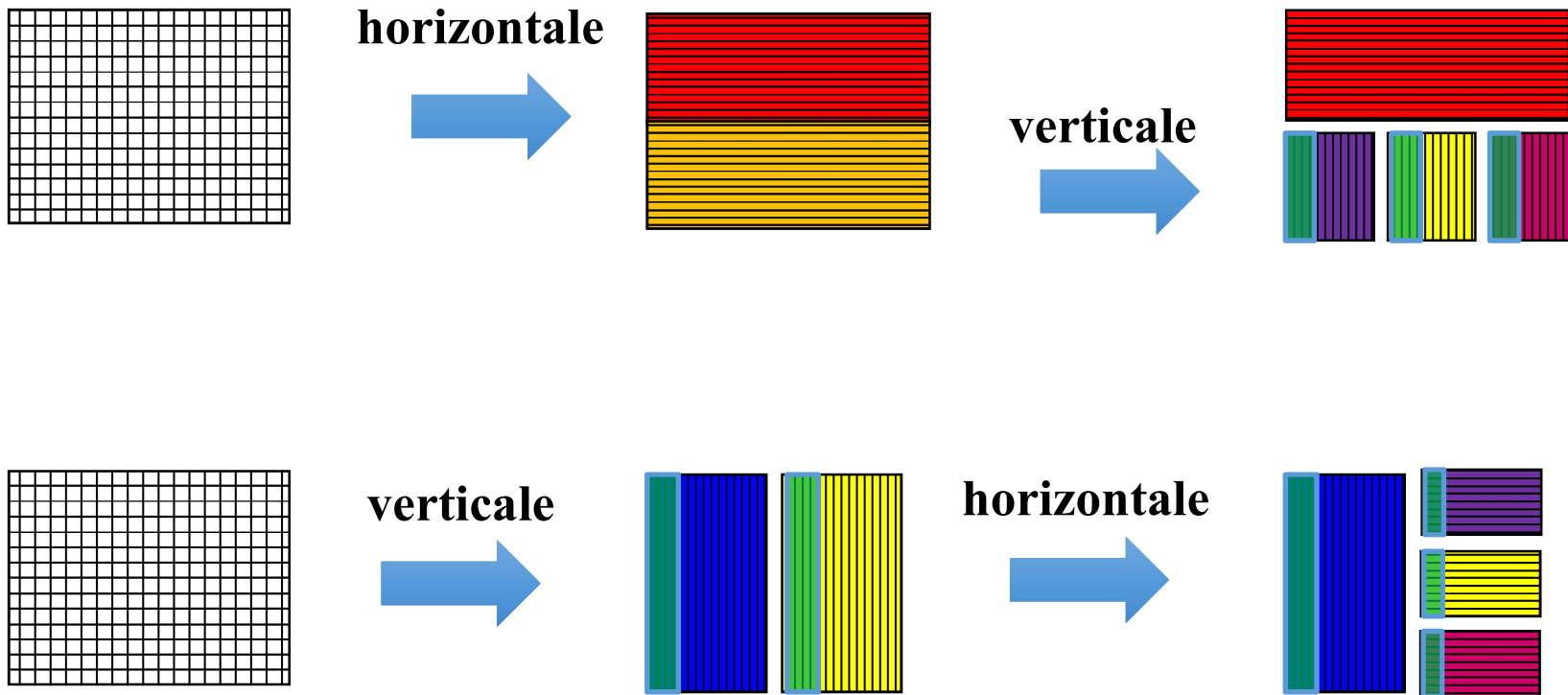
horizontale dérivée

verticale



Clé = attribut commun

Fragmentation Mixte



Fragmentation correcte

Complète

- chaque élément de R doit se trouver dans un fragment

Reconstructible

- on doit pouvoir recomposer R à partir de ses fragments (ressemble à décomposition de schéma vue en BD1 pour fragmentation verticale)

[Disjointe] /*si on veut éviter réplication pour cohérence */

- chaque élément de R ne doit pas être dupliqué (sauf clé en cas de fragmentation verticale)

En pratique : seulement Complète et Reconstructible, mais pas disjointe.

Fragmentation Horizontale

Fragments définis par sélection

$\text{Client}_1 = \sigma_{\text{ville} = \text{'Dakar'}} \text{Client}$

$\text{Client}_2 = \sigma_{\text{ville} \neq \text{'Dakar'}} \text{Client}$

Inférence : correcte

Reconstruction par union

$\text{Client} = \text{Client}_1 \cup \text{Client}_2$

En SQL :

create view Client as

select * from Client₁

union

select * from Client₂

Client

nclient	nom	ville
C 1	Diop	Dakar
C 2	Marega	Thies
C 3	Kambe	Dakar
C 4	Sarr	Tamba

Client1

nclient	nom	ville
C 1	Diop	Dakar
C 3	Kambe	Dakar

Client2

nclient	nom	ville
C 2	Marega	Thies
C 4	Sarr	Tamba

Fragmentation Horizontale : Exemples

Fragments définis par sélection d'intervalles

Ville(numV, nom, cp, population)

$V_1 = \sigma_{\text{population} < 10K} \text{ Ville}$

$V_2 = \sigma_{10K \leq \text{population} < 50K} \text{ Ville}$

$V_3 = \sigma_{50K \leq \text{population}} \text{ Ville}$

Fragments définis par sélection sur plusieurs attributs

Personne(numP, nom, prenom, age, statut)

$P_1 = \sigma_{\text{age} < 18 \text{ and statut} = \text{élève}} \text{ Personne}$

$P_2 = \sigma_{\text{age} < 18 \text{ and statut} = \text{employé}} \text{ Personne}$

$P_3 = \sigma_{\text{age} \geq 18} \text{ Personne}$

Fragmentation Horizontale par sélection

- Fragmentation de R selon n prédicats
 - les prédicats $\{p_1, \dots, p_n\}$ ex: $\{a < 10, a > 5, b = 'x', b = 'y'\}$
- L'ensemble M des prédicats de fragmentation est :
 - $M = \{m \mid m = \bigwedge_{1 \leq k \leq n} p_k^* \}$ avec $p_k^* \in \{p_k, \neg p_k\}$
 - **Eliminer les m de sélectivité nulle** ex: $a > 10 \wedge a < 5$
 - **Simplifier**: $a < 10 \wedge a \leq 5 \wedge b = 'x' \wedge b \neq 'y'$ devient $a \leq 5 \wedge b = 'x'$
- Construire les fragments $\{R_1, \dots, R_k\}$
 - Pour chaque m_i , $R_i = \sigma_{m_i}(R)$
- Minimalité
 - Ne pas avoir 2 fragments toujours lus ensemble
- Choisir les p_i des requêtes les plus fréquentes
- Ref bibliographique sur la fragmentation et le choix des fragments optimaux.
Regroupement hiérarchique d'ensembles de nuplets issus de requêtes fréquentes.
 - Waterloo Univ : G. Aluç, M. T. Özsu, K. Daudjee and O. Hartig.
 - "Executing Queries over Schemaless RDF Databases", ICDE 2015 (Int'l Conf. on Data Engineering)

Fragmentation Horizontale Dérivée

Fragments définis par **semi jointure**

```
Cde1 = select Cde.*  
      from Cde, Client 1  
      where Cde.nclient = Client1.nclient
```

$Cde_i = Cde \bowtie Client_i$ pour i dans $\{1; 2\}$

Reconstruction par union

$Cde = Cde_1 \cup Cde_2 = \bigcup_i Cde_i$

Cde

ncde	nclient	produit	qté
D 1	C 1	P 1	10
D 2	C 1	P 2	20
D 3	C 2	P 3	5
D 4	C 4	P 4	10

Cde1

ncde	nclient	produit	qté
D 1	C 1	P 1	10
D 2	C 1	P 2	20

Cde2

ncde	nclient	produit	qté
D 3	C 2	P 3	5
D 4	C 4	P 4	10

Fragmentation Horizontale Dérivée

Méthode générale

- Commencer par recenser les fragments horizontaux déjà définis
- Repérer des clés étrangères dans la relation à fragmenter
- Semi-jointure(s)

Exemple avec 2 semi-jointures

- Personne(numP, nom, premon, age)
 - 4 fragments pour a=[0,18,25,60,100]
 - $P_i = \sigma_{a_{i-1} \leq \text{age} < a_i} \text{Personne}$
- Sport(numS, type)
 - 2 fragments pour t=[indiv, collectif]
 - $S_j = \sigma_{\text{type} = t_j} \text{Sport}$
- Inscrit(numP*, sport*, date)
 - $4 * 2 = 8$ fragments: $\text{Inscrit}_{ij} = \text{Inscrit} \bowtie P_i \bowtie S_j$

Propriétés de la fragmentation horizontale dérivée

R: fragmentation horizontale $\rightarrow$ fragments R_i

S: fragmentation horizontale dérivée $\rightarrow$ fragments $S_i = S \bowtie_A R_i$

- Complète

- Chaque tuple de S doit joindre avec au moins un tuple de R
 - $\forall s \in S, \exists t \in S_i, s = t$

- Disjointe

- $\forall i, j \text{ tq } i \neq j, S_i \cap S_j = (S \bowtie R_i) \cap (S \bowtie R_j) = S \bowtie (R_i \cap R_j) = \emptyset$
 - Rappel: $R_1 \cap R_2 \Leftrightarrow R_1 \bowtie R_2$

- Reconstructible

- $\bigcup_i S_i = (S \bowtie R_1) \cup (S \bowtie R_2) \cup \dots \cup (S \bowtie R_n) = S \bowtie (\bigcup_i R_i) = S$

- $\Rightarrow$ contrainte d'intégrité référentielle

- A = clé de R
- S.A référence R.A
 - $\forall s \in S, \exists r \in R, s.A = r.A$

Fragmentation **Verticale**

Fragments définis par **projection**

$Cde1 = \pi_{ncde, nclient} Cde$

$Cde2 = \pi_{ncde, produit, qté} Cde$

Cde

ncde	nclient	produit	qté
D 1	C 1	P 1	10
D 2	C 1	P 2	20
D 3	C 2	P 3	5
D 4	C 4	P 4	10

Reconstruction par jointure

$Cde = Cde1 \bowtie Cde2$

En SQL :

create view Cde as

select * from Cde₁, Cde₂

where Cde₁.ncde = Cde₂.ncde

Cde1

ncde	nclient
D 1	C 1
D 2	C 1
D 3	C 2
D 4	C 4

Cde2

ncde	produit	qté
D 1	P 1	10
D 2	P 2	20
D 3	P 3	5
D 4	P 4	10

Fragmentation Verticale

Comment définir une fragmentation verticale ?

- Affinité des attributs : mesure la proximité « *sémantique* » des attributs (combien « ils vont ensembles »)
 - Soit par connaissance de l'application,
 - soit par analyse des requêtes (on mesure combien de fois deux attributs donnés ont été interrogé ensembles)
 - Résultat sous forme de matrice d'affinité (TPE)
- 2 approches : regroupement, partitionnement (grouping – splitting)
 - Regroupement -> attributs vers fragments
 - partitionnement -> relation vers fragments
- Algorithme de regroupement des attributs bien adapté (clustering)
 - BEA : bond energy algorithm (Mc Cormick et al. 72) : $O(n^2)$ (lecture TPE)
 - insensible à l'ordre de départ des attributs
 - Part des attributs individuels et effectue des regroupements de groupes
- Algorithme de partitionnement :
 - Part d'une relation et observe le bénéfice qu'on peut tirer à partitionner

Matrice d'affinité des attributs

Matrice A

a_{ij} = affinité de A_i avec A_j

ex: nb de requêtes qui accèdent A_i et A_j

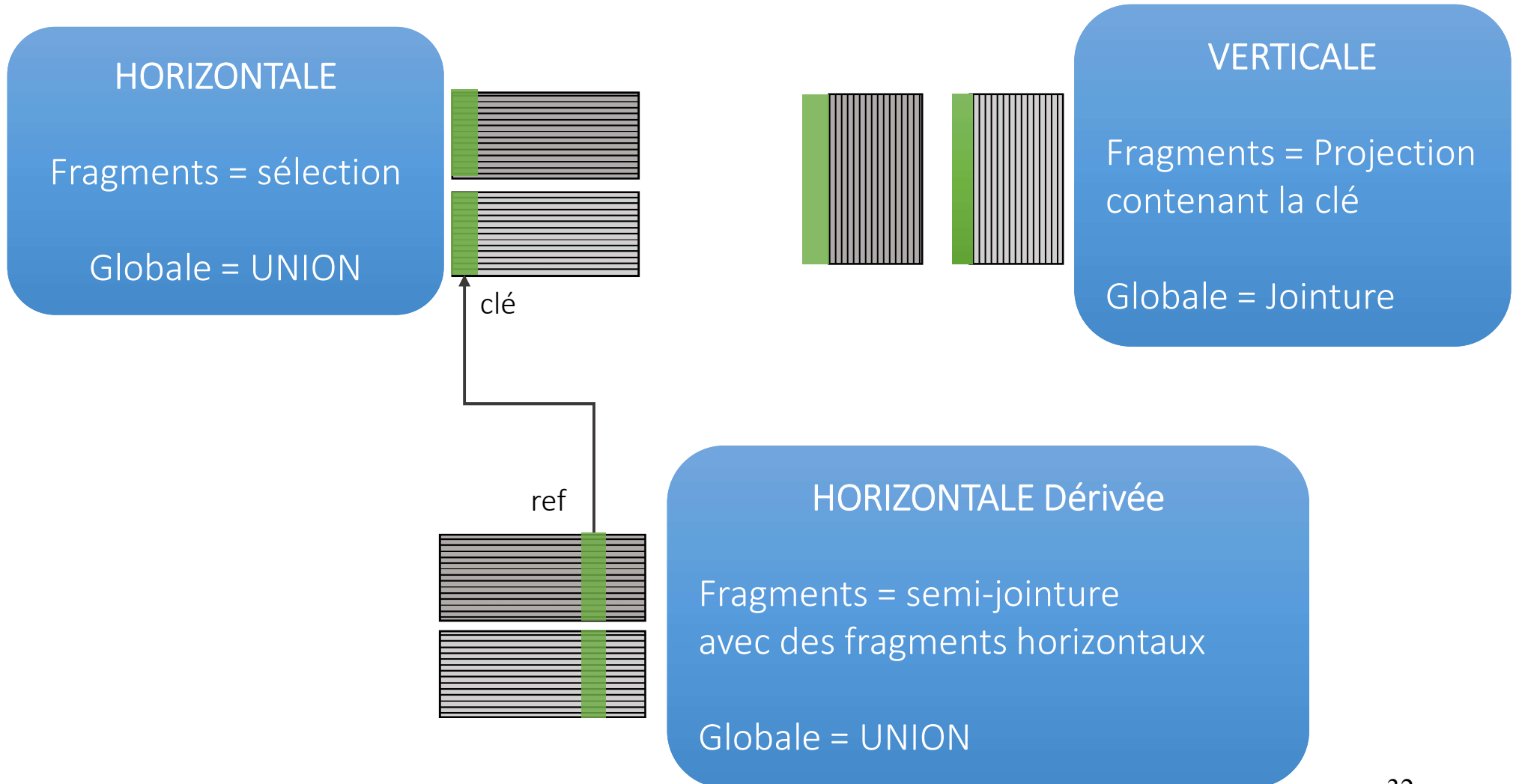
	A1	A2	A3	A4
A1	45	0	45	0
A2		80	5	75
A3			53	3
A4				78

Matrice d'affinité regroupement des attributs

Matrice A

	A1	A3	A2	A4
A1	45	45	0	0
A3		53	5	3
A2			80	75
A4				78

Résumé des Fragmentations



Allocation des Fragments aux Sites

Non-répliquée

- partitionnée : chaque fragment réside sur un seul site

Dupliquée

- chaque fragment sur un ou plusieurs sites
- maintien de la cohérence des copies multiples : coûteux
- **(le fameux) Compromis Lecture/écriture:**
 - + le ratio Lectures/màj est > 1 , + la duplication est avantageuse

Allocation de Fragments

Problème: Soit

F un ensemble de fragments

S un ensemble de sites

Q un ensemble d'applications et leurs caractéristiques

trouver la distribution "optimale" de F sur S

Optimum

- coût minimal de communication, stockage et traitement
- Performance = temps de réponse ou débit

Solution

- allouer une copie de fragment là où le bénéfice est supérieur au coût

Exemple d'Allocation de Fragments

Client1

nclient	nom	ville
C 1	Diop	Dakar
C 3	Kambe	Dakar

Client2

nclient	nom	ville
C 2	Marega	Thies
C 4	Sarr	Tamba

Cde1 = Cde ▶< Client1

ncde	client	produit	qté
D 1	C 1	P 1	10
D 2	C 1	P 2	20

Site 1

Cde2

ncde	client	produit	qté
D 3	C 2	P 3	5
D 4	C 4	P 4	10

Site 2

Exemple : Sport universitaire

Trois universités Dakaroises (UCAD, UVS, UAM) ont décidé de mutualiser leurs équipements sportifs (locaux) et les entraîneurs. La gestion commune est effectuée par une **base de données répartie**, dont le schéma **global** est le suivant :

PROF (Idprof, nom, adresse, tél, affectation, salaire)

ETUDIANT (Idetu, nom, adresse, assurance, police, université, équipe)

LOCAUX (Idlocal, adresse, université)

EQUIPE (équipe, sport, niveau)

HORAIRE (Idlocal, équipe, jour, heure_début, heure_fin, prof)

- Chaque université rémunère ses profs en envoyant un chèque à leur adresse, mais aussi elle doit pouvoir contacter tout prof qui utilise ses locaux.
- Chaque équipe correspond à un sport. La plupart des équipes ont droit à un (seul) créneau (jour, heure) dans un des locaux communs pour leur entraînement. Cependant, pour le sport «cyclisme », il n'y pas besoin de locaux.
- Chaque université gère évidemment ses propres étudiants, ainsi que ses locaux et les créneaux correspondants.
- Les équipes ne sont associées à aucune université en particulier. Cependant, pour des questions d'assurance, chaque université doit aussi gérer les étudiants qui utilisent ses locaux. Pour le cyclisme , c'est l'UAM qui en a la charge.
- Les relations **globales** sont **fragmentées et réparties sur les différents sites**.

Exemple : Sport universitaire : Une solution

- $a=[UCAD, UVS, UAM]$ i dans $\{1 ; 2; 3\}$
- $Locaux_i = \sigma_{univ= a_i} (Locaux)$
- $Horaire_i = Horaire \bowtie Locaux_i$

Fragmentation mixte pour les profs :

- $ProfPaye_i = \pi_{idProf, nom, adresse, salaire} (\sigma_{affectation = a_i} (Prof))$
- $ProfContact_i = \pi_{idProf, nom, tel} (Prof \bowtie Horaire_i)$

Etudiant fragmenté par université et on rajoute les étudiant qui utilisent les équipements

- $Etu_i = (\sigma_{affectation = a_i} (Etudiant)) \cup (Etudiant \bowtie (Horaire \bowtie Locaux_i))$
- Pour i dans $\{1 ; 2\}$ on a $Etudiant_i = Etu_i$
- Pour $i=3$, il faut rajouter $EtuCycle = Etu \bowtie (\sigma_{sport = "cyclisme"} Equipe)$
- $Etudiant_3 = Etu_3 \cup EtuCycle$
- Equipe est répliquée partout (petite et très rarement mise à jour)

Conclusions et perspectives

Applications classiques

- décisionnel (data warehouse)
- transactionnel

Applications à l'échelle du web

- grand nombre de sources
- hétérogénéité très forte
- intégration des données semiestructurées
- intégration de la recherche documentaire
- intégration de services Web (ex. agence de voyage)